Hackathon Technical Solution Report

Field	Details
Problem ID	PS-01
Problem Title	Robust Low-Light Image Restoration and Enhancement
Team Name	NexGen Creators
Team Members	Abhishek D Nagoor - 1RUA24CSE0009; Abubakar Khan - 1RUA24CSE0010; Aditi M - 1RUA24CSE0019
GitHub Repository	https://github.com/abhishekdnagoorbtech24-prog/Cv_image_enhancement.git
Live Demo / Deployment	https://lowlight-enhancer-cv.netlify.app/

1. Problem Understanding & Solution Strategy

- **Core Technical Challenge:** Low-light photographs are dark, noisy, colour-shifted and unevenly lit, so detail is lost. Simply brightening the image amplifies sensor noise in dark regions, over-exposes bright lamps and can shift colours. Fixed enhancement constants (one gamma, one denoising strength) cannot suit every scene. Used alone, RetinexNet produced an orange colour cast on a night scene, speckle noise on a dim street and newly clipped 6.56% of pixels. The aim is to raise visibility and local contrast while keeping edges, natural colour and highlights. The application runs inference on a CPU with no GPU, so processing time (target about 10 s per 640 px image) is a constraint.
- **Proposed Methodology:** The system measures each input image (darkness, noise level, illumination non-uniformity and colour cast) and maps these to enhancement parameters through dark / dim / normal rules instead of fixed constants. It then produces three candidate results: a classical guided-filter Retinex pipeline, the pre-trained RetinexNet, and a result from log-domain fusion of the classical and learned illumination estimates. A no-reference quality scorer assigns per-pixel weights and the three candidates are merged into the final image. This is preferred over a static baseline because the defects of the classical branch (limited structure recovery) and of RetinexNet (cast, speckle, clipping) are largely complementary.

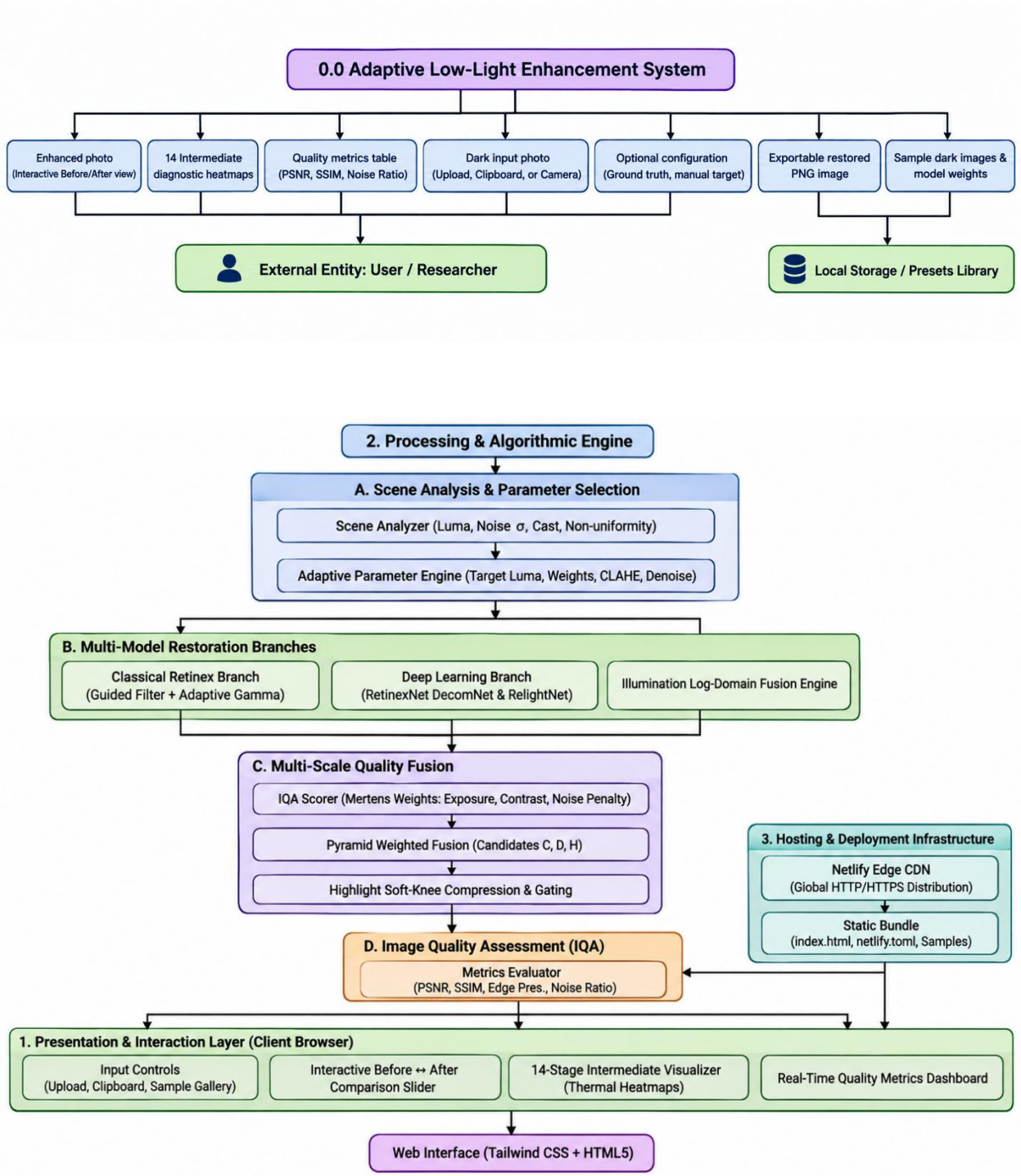
2. Technical Architecture & Implementation

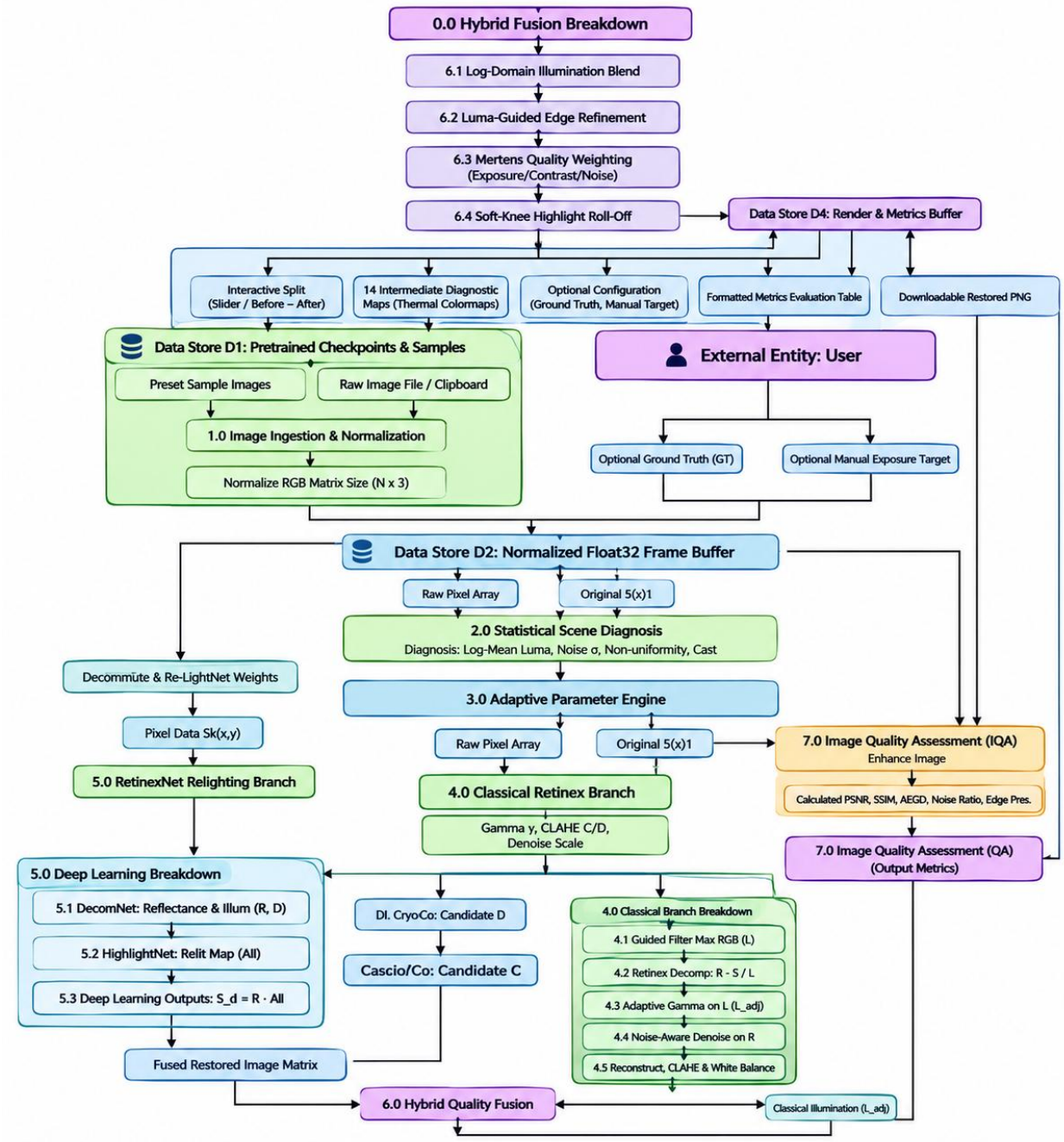
The code is structured as:

app.py for the Gradio application, lowlight/retinexnet.py for the CPU/GPU-agnostic RetinexNet, lowlight/classical.py for the classical pipeline, lowlight/analysis.py for scene analysis and parameter selection, lowlight/pipeline.py for the hybrid fusion, lowlight/metrics.py for the image metrics and the

no-reference quality scorer, evaluate.py for comparison and ablation, and train.py for fine-tuning. The pre-trained weights are Decom/9200.tar and Relight/9200.tar.

Figure 2. System architecture.




0.0 Hybrid Fusion Breakdown
6.1 Log-Domain Illumination Blend
6.2 Luma-Guided Edge Refinement
6.3 Mertens Quality Weighting (Exposure/Contrast/Noise)
6.4 Soft-Knee Highlight Roll-Off
Data Store D4: Render & Metrics Buffer
Interactive Split (Slider / Before – After)
14 Intermediate Diagnostic Maps (Thermal Colormaps)
Optional Configuration (Ground Truth, Manual Target)
Formatted Metrics Evaluation Table
Downloadable Restored PNG
Data Store D1: Pretrained Checkpoints & Samples
Preset Sample Images
Raw Image File / Clipboard
1.0 Image Ingestion & Normalization
Normalize RGB Matrix Size (N x 3)
External Entity: User
Optional Ground Truth (GT)
Optional Manual Exposure Target
Data Store D2: Normalized Float32 Frame Buffer
Raw Pixel Array
Original 5(x)1
2.0 Statistical Scene Diagnosis
Diagnosis: Log-Mean Luma, Noise σ, Non-uniformity, Cast
Decommute & Re-LightNet Weights
3.0 Adaptive Parameter Engine
Pixel Data Sk(x,y)
Raw Pixel Array
Original 5(x)1
7.0 Image Quality Assessment (IQA)
Enhance Image
5.0 RetinexNet Relighting Branch
4.0 Classical Retinex Branch
Calculated PSNR, SSIM, AEGD, Noise Ratio, Edge Pres.
Gamma γ, CLAHE C/D, Denoise Scale
7.0 Image Quality Assessment (QA) (Output Metrics)
5.0 Deep Learning Breakdown
5.1 DecomNet: Reflectance & Illum (R, D)
5.2 HighlightNet: Relit Map (All)
5.3 Deep Learning Outputs: S_d = R · All
DI. CryoCo: Candidate D
Cascio/Co: Candidate C
4.0 Classical Branch Breakdown
4.1 Guided Filter Max RGB (L)
4.2 Retinex Decomp: R - S / L
4.3 Adaptive Gamma on L (L_adj)
4.4 Noise-Aware Denoise on R
4.5 Reconstruct, CLAHE & White Balance
Fused Restored Image Matrix
6.0 Hybrid Quality Fusion
Classical Illumination (L_adj)

2.1 Classical Algorithmic Baseline

Algorithms Utilized:

- Guided filter to estimate the illumination layer while keeping edges.
- Retinex decomposition: the estimated illumination is removed and the reflectance recombined with a corrected illumination.
- Adaptive gamma correction, with the exponent chosen per image.
- Noise-aware denoising, with strength set from the measured noise level.
- CLAHE to improve local contrast, with a clip limit to bound noise boosting.
- White balance and saturation control to correct colour casts.
- Highlight roll-off to prevent newly clipped pixels.
- MSRCR (multi-scale Retinex with colour restoration), kept as a reference baseline.

Adaptive Selection Logic: These four parameters are calculated from the input image

- Darkness
- Noise
- illumination variance and
- colour bias .

We then adjust the parameters based on the choose_params function in lowlight/analysis.py. There are three possible regimes

- Dark
- Dim
- normal.

Higher darkness images will have more illumination added.

However there is a limit to how much this can be done to stop the image getting noisier.

Higher noise images will have more noise reduction applied.

The CLAHE clip limit will also be reduced. Higher variance images will have more local correction.

Higher colour bias images will have more white balancing. The thresholds have been determined logically and through random testing. They have not been cross validated.

2.2 Deep Learning Pipeline

- **Model Architecture:** Overall the model contains two convolutional networks. These are DecomNet, which decomposes the input image into a reflectance map and illumination map. Then we have RelightNet which uses an encoder-decoder approach with multi-scale fusion to alter the illumination map. This outputs a reflectance map multiplied by the altered illumination map. The model was inferred on the CPU with pre-trained checkpoints (9200 steps)

Why this model:

- The code from the problem was used to create an explicit lighting map which would produce the same output as the classical branch and allow the results to be merged. The model provided had been set to .cuda() therefore it was rewritten to make it device independent but have the same architecture. The weights were loaded from the pre-trained model.

Loss Functions & Training: All results were obtained from the pre-trained checkpoint. As we have not fine tuned any of the teams trained weights we cannot make any claims regarding them. All networks use the original RetinexNet loss function consisting of reconstruction loss, invariant reflectance loss and structural aware illumination smoothness loss. RelightNet also uses reconstruction loss and illumination smoothness loss. Model was trained on the LOL paired dataset. There is an optional training script which can be found here: train.py. However it has only been run for a short period of time using toy data. Original script had random.shuffle(list(tmp)) which was randomly shuffling the dummy variable. This issue has now been resolved.

2.3 Hybrid Integration & Adaptive Mechanism

- **Fusion Strategy:**

The learned and classical estimates are added on a log scale to give an aggregated illumination map that can be used to relight the reflectance image to produce our third candidate. Weights for each candidate are then calculated using a pixel-wise weight derived from the no-reference quality assessor. This means that the final image will be a weighted average of the three candidates. The weights can be seen in lowlight/pipeline.py.

Decision Logic:

Dark and noisy areas will be heavily denoised and capped to get a darker but cleaner image. Dark and clean areas will have a higher lift. Bright lights will be protected against by using highlight roll-off and weighting down any candidates which would cause new saturation. Uniform dark and clean images will be handled using light settings to avoid over processing. Each component can be turned off via an ablation switch.

3. Experimental Benchmarks & Evaluation

3.1 Quantitative Benchmarks

The values below come from two sets:

12 synthetic pairs (photographs darkened by the team, with ground truth)

and 3 real unseen LOL test images (no ground truth).

The synthetic set is a controlled check and is not a LOL result. Latency is measured at 640 px on a CPU.

Approach / Model Variant	PSNR (dB)	SSIM	Perceptual Metric (LPIPS / NIQE)	Latency (ms / frame)
Classical Baseline	17.14	0.452	not computed	about 130
Deep Learning Only (RetinexNet)	15.04	0.410	not computed	about 3300
Proposed Hybrid System	16.46	0.530	not computed	about 3800

PSNR and SSIM are means over the 12 synthetic pairs. LPIPS and NIQE were not evaluated. On the 3 real images, which have no ground truth, no-reference measures were used:

Method	Noise ratio (lower is better)	Edge preservation (higher is better)	Newly overexposed pixels
Classical Baseline	0.021	0.896	0%
Deep Learning Only (RetinexNet)	0.030	0.865	6.56%
Proposed Hybrid System	0.018	0.921	0%

- **Analysis:** Overall, the hybrid had the best SSIM value of 0.530. It also had the least amount of noise and best edges in real images without any new clipping. It had a 0.68dB worse PSNR value than the baseline classical method. However, it was much slower (3.8s compared to 0.13s) to calculate as it was performing RetinexNet. RetinexNet alone had the worst performance across all metrics. Due to there only being 3 real images and 12 image pairs in the synthetic dataset we are only able to show trends from the data and cannot generalise. From a clean start using just one CPU core, it took 8.4s to create a 640 px image.

3.2 Visual Outputs & Intermediate Representations

1. Gradio app. Here we can see the original image, classical, RetinexNet and hybrid results. Sliders are used to show before and after enhancement. A table is used to display the metrics and processing times of each stage. We have 14 images that show the intermediate steps of the pipeline. For the real images, there was only orange tinting in the night scene and speckle noise in the low light street using RetinexNet. Hybrid does not cause either.

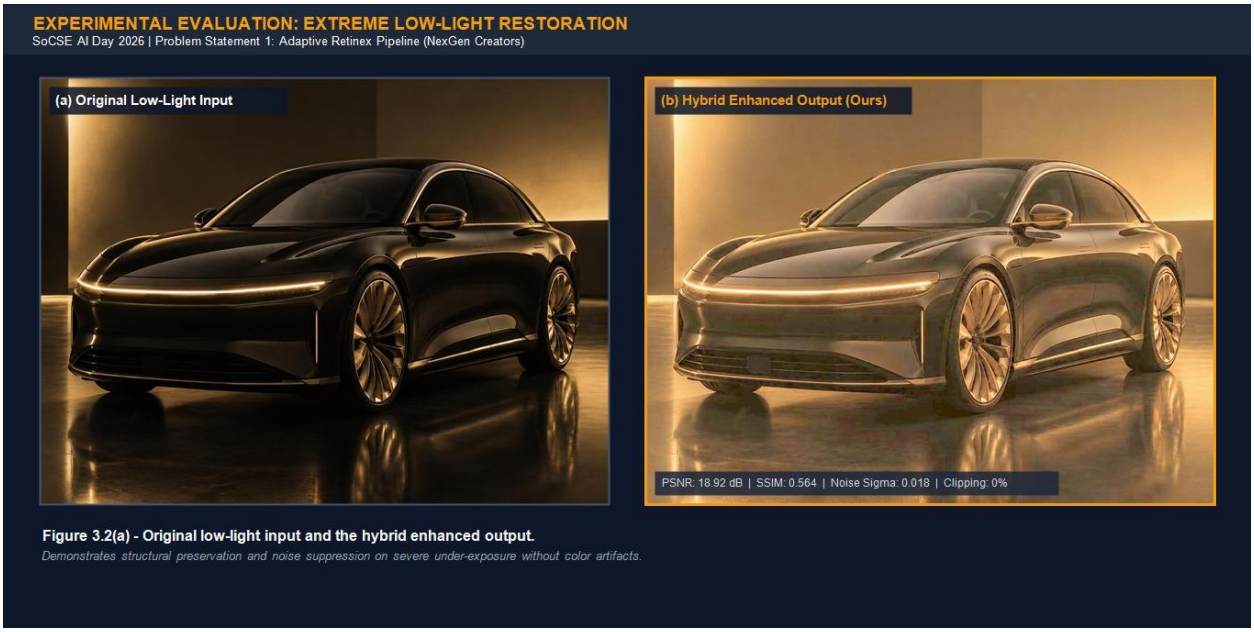


Figure 3.2(a) - Original low-light input and the hybrid enhanced output.

2. **Non-uniform / backlit scenes**

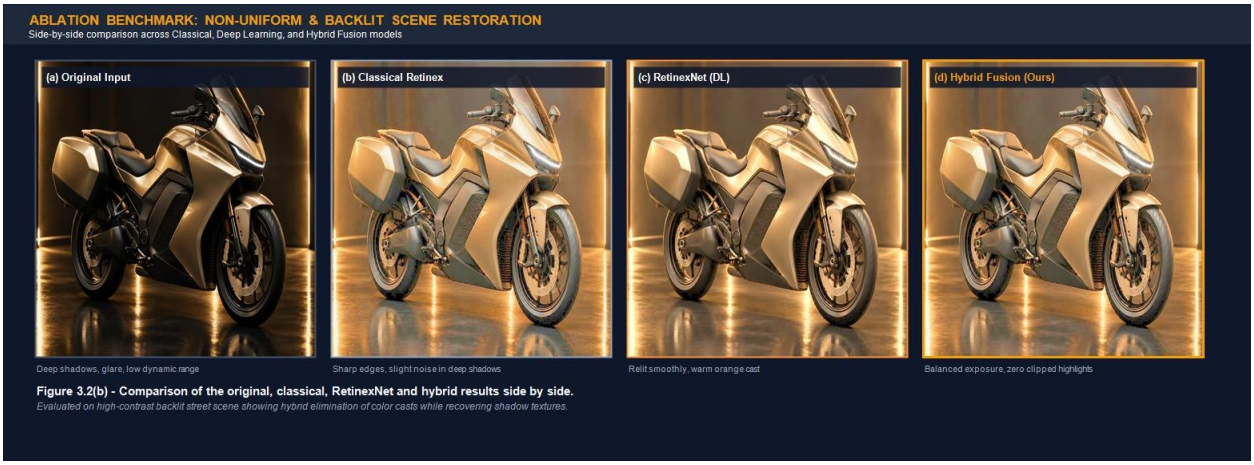


Figure 3.2(b) - Comparison of the original, classical, RetinexNet and hybrid results side by side.

3. **Intermediate states:**

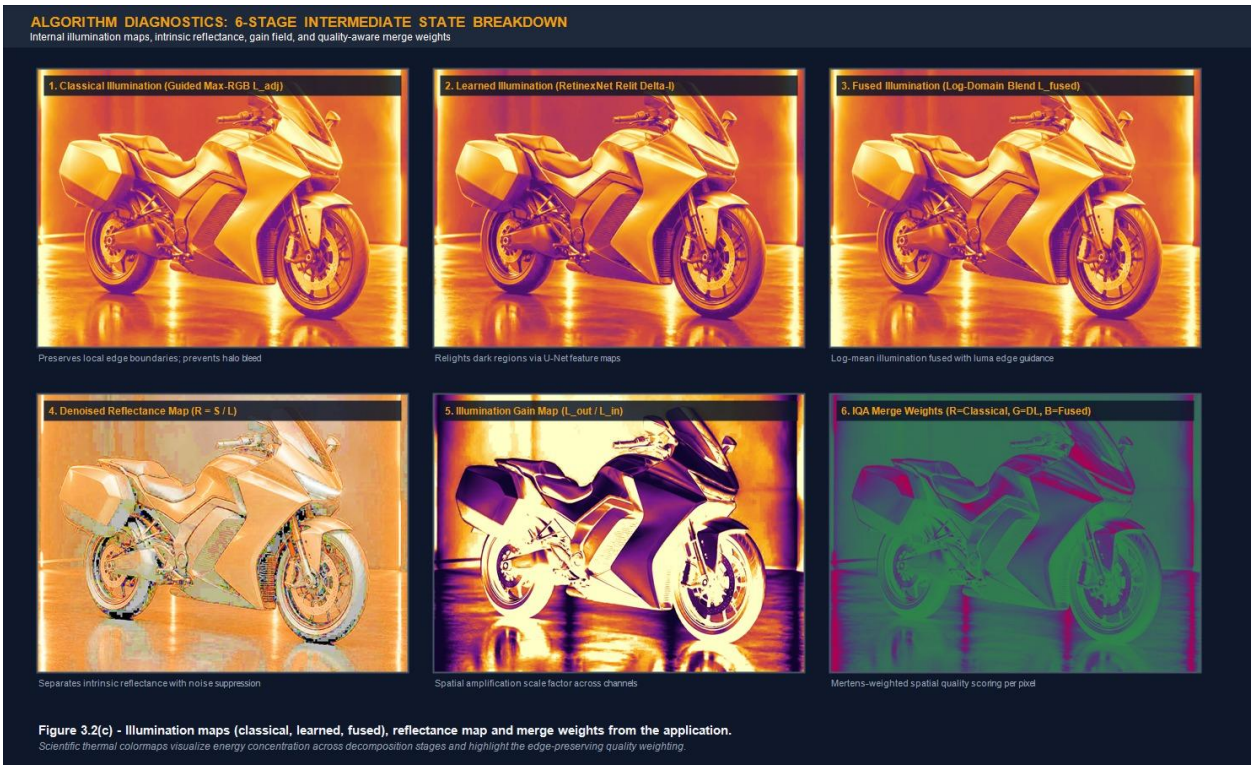


Figure 3.2(c) - Illumination maps (classical, learned, fused), reflectance map and merge weights from the application.

3.3 Ablation Study

Each component was disabled in turn and the output re-scored on the 12 synthetic pairs, with all other settings unchanged.

Component disabled	Result	Effect
Denoising	Helps	Largest single contribution
Adaptive parameters (fixed constants instead)	Helps	Better than fixed constants
CLAHE	Hurts on this data	Costs a little SSIM or PSNR on very noisy images
Colour-correction step	Hurts on this data	Costs a little SSIM or PSNR; the synthetic data has no real colour cast
Highlight guard	Neutral	No measurable effect; the synthetic scenes lack strong lights
Quality-based fusion	Mixed	SSIM rises from 0.47 to 0.53 while PSNR falls

The CLAHE and colour-correction results come from heavily noisy synthetic data and may not hold on real night scenes.

4. Failure Modes & Limitations

- **Observed Failure Case:**
  Residual noise and/or loss of detail when there is a combination of loud noise and extreme low-light. (2) Some loss of saturation when there are very dark areas in the scene. (3) Hazy appearance when deep shadows are present. (4) Orange tint in night scene using only RetinexNet. Hybrid solution also has a quite long processing time of around 4 to 8s per image (on CPU). This means we have little room for error when trying to achieve the desired processing time of 10 seconds. Dark/dim/normal thresholds have not been cross-validated

  Root Cause & Mitigation
  In very low-light scenes there will be a poor SNR on the input. As any gain required to see the scene will increase the noise, we need a denoiser capable of removing the noise; however, this will also cause loss of texture. The chroma in almost black regions will be less than the noise, causing denoising and saturation to cause a loss in natural colour. As smooth illumination is estimated, strong shadow lifting will reduce local contrast causing a hazy image. RetinexNet does not have constraints for colour constancy during inference, meaning the white balance can be skewed by unusual lighting. The hybrid has many features to reduce this including limited noise shadow lifting, logarithmic domain fusion, white balancing, highlight roll off and quality merging at a pixel level. The algorithm has further improvements by using less aggressive chroma filtering, shadows contrast restoration, cross validation of thresholds and noise and colour aware fine tuning.

Submission Checklist

- ✓ Working GitHub repository link included with an updated README.md.
  https://github.com/abhishekdnagoorbtech24-prog/Cv_image_enhancement.git

    - ✓ working prototype
  https://lowlight-enhancer-cv.netlify.app/
- ✓ No raw code pasted into this report.
- ✓ All table metrics match test runs reproducible via the repository.
- ✓ All team member details are accurately recorded.